Blob Detection

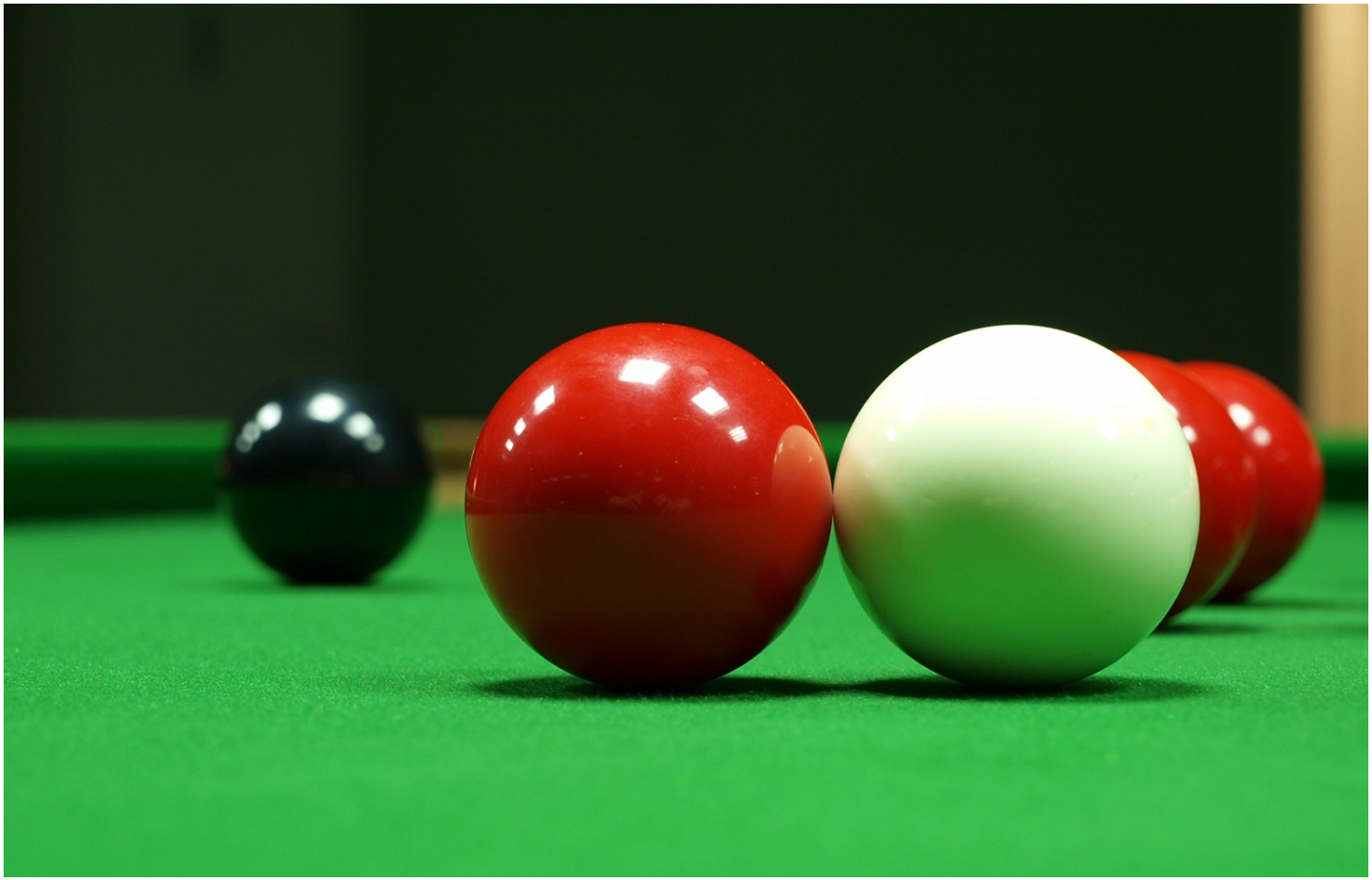
Thumrongsak Kosiyatrakul

Blob

- A **blob** in computer vision is a region of image that some properties is constant or approximately constant
- Example of properties:
  - Color
  - Brightness
- A robot can detect an object with a solid color by simply find blobs of that color
- A blob usually defined as a rectangular region with information:
  - Width and Height
  - Number of pixels
  - Coordinate of the center of the region

How to Detect a Blob?

- Consider the following image:



- How to detect a red ball in the above image?

Filter Selected Color

- Image generally comes in RGB format
- Each pixel is represented by three values, red, green, and blue
- To filter out a specific color, we simply define ranges for each value
  - Red between 120 to 130,
  - Green between 78 to 90, and
  - Blue between 100 to 110

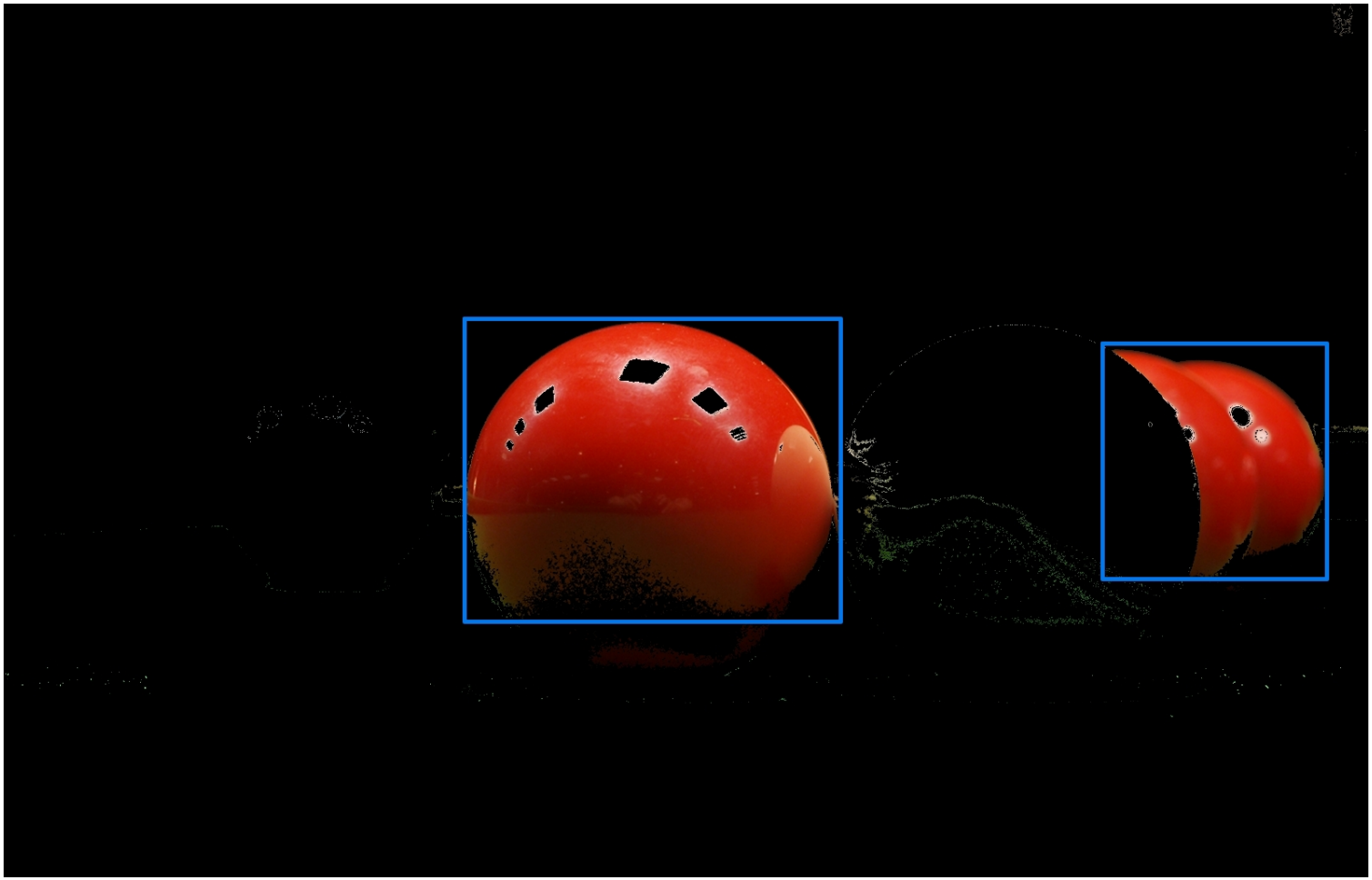
How to Detect a Blob

- Filter out every color but red



How to Detect a Blob

- Detect connected regions



Filter Selected Color

- Note that an object generally stays in a foreground where environment is in the background
- Foreground objects generally have more brightness
- In computer vision, we prefer YUV format:
  - Y stands for the luminance component (the brightness)
  - U and V are the chrominance (color) components
- Conversion between RGB and YUV is straightforward:
  - $Y = (0.299 \times R) + (0.587 \times G) + (0.114 \times B)$
  - $U = (-0.168736 \times R) + (-0.331264 \times G) + (0.5 \times B)$
  - $V = (0.5 \times R) + (-0.418688 \times G) + (-0.081312 \times B)$

  where R, G, and B have values between 0.0 and 1.0

Problems with Blob Detection

- If we have two objects with the same color, **at least** two blobs will be detected
- If we have only one object, more than one blobs may be detected
  - Lighting effects color
  - Filtered regions may not connect with each other
    - Two or more blobs but on the same object
    - If all blobs come from the same object, they will be close to each other
  - Surrounding environment may have the same color
- Same object may not be detected if lighting change
- Increase the range of color may result in detecting environment instead of desire object
- Only works in restricted lighting and surrounding environment has small number of colors

Applications

- Object Recognition and Tracking:
  - Soccer: Ball and goal posts
  - Navigation: Follows a line
  - Object avoidance
- Robot Location/Orientation:
  - Two colors on top of robot
  - Signs (multi-colors) represent locations

Integrated Web Camera

- To detect blobs, we need an image or a series of images
- We will use integrated Web camera of your given laptop
- To access the web cam, we need another node called `gscam`
  - `gscam` node communicate with integrated Web cam
  - Capture video and turn them into series of images
  - Each image will be published on the topic

```
/v4l/camera/image_raw
```

  - To start `gscam` node, use the following command:

```
roslaunch gscam v4l.launch
```

- A series of images can be viewed using `image_view` node as follows:

```
rosrun image_view image_view image:=/v4l/camera/image_raw
```

`Image` Message

- Images published by `gscam` node has type `sensor_msg/Image.msgs`
- If you want to access images, add the following:

```
from sensor_msg.msgs import Image
```

- The `Image` message consists of the following components:
  - `header`: Information about message
  - `height`: Image height (number of rows)
  - `width`: Image width (number of columns)
  - `encoding`: Type of encoding
  - `is_bigendian`: Is this data bigendian
  - `step`: How many bytes in a row
  - `data`: A one-dimensional array of data. The number of elements is `step * rows` (`[B,G,R,B,G,R,...]`)
- Lucky us, we do not have to process images directly.

View Image from Camera using OpenCV

- We can create our own image viewer from camera using OpenCV
- First, we need to convert a color image message into OpenCV format
- Then, simply display the image
- Code Example:

```
#!/usr/bin/env python
import rospy
import cv2
from sensor_msgs.msg import Image
from cv_bridge import CvBridge, CvBridgeError

colorImage = None
isColorImageReady = False

def updateColorImage(data):
    global colorImage, isColorImageReady
    colorImage = data
    isColorImageReady = True
```

View Image from Camera using OpenCV

- Code Example (continue):

```
def main():
    global colorImage, isColorImageReady
    rospy.init_node('myimageview', anonymous=True)
    rospy.Subscriber('/v4l/camera/image_raw', Image, updateColorImage)
    bridge = CvBridge()
    cv2.namedWindow('Image from Camera')

    while not rospy.is_shutdown() and not isColorImageReady:
        pass

    while not rospy.is_shutdown():
        try:
            color_image = bridge.imgmsg_to_cv2(colorImage, "bgr8")
        except CvBridgeError, e:
            print e

        cv2.imshow('Image from Camera', color_image)
        key = cv2.waitKey(1)

        if key == ord('q'):
            break

    cv2.destroyAllWindows()

if __name__ == '__main__':
    main()
```

`cmvision` Node

- `cmvision` node detects blobs of colors from `Image` message
  - Subscribes to a topic (of type `Image`) of your choice
  - In our case, the topic will be:

    ```
    /v4l/camera/image_raw
    ```

  - Obtains user defined color information about blobs to be detected
  - Publishes information about detect blobs on the topic `/blobs`
- `cmvision` package also supplies a tool named `colorgui`
  - Allows user to pick a range of colors of an object of interest
  - Shows visual image of detect blobs associated with picked color
  - Supplies color information and threshold

- Before running `colorgui` node, make sure your `roscore` and `gscam` is running
- To run `colorgui`, use the following command:

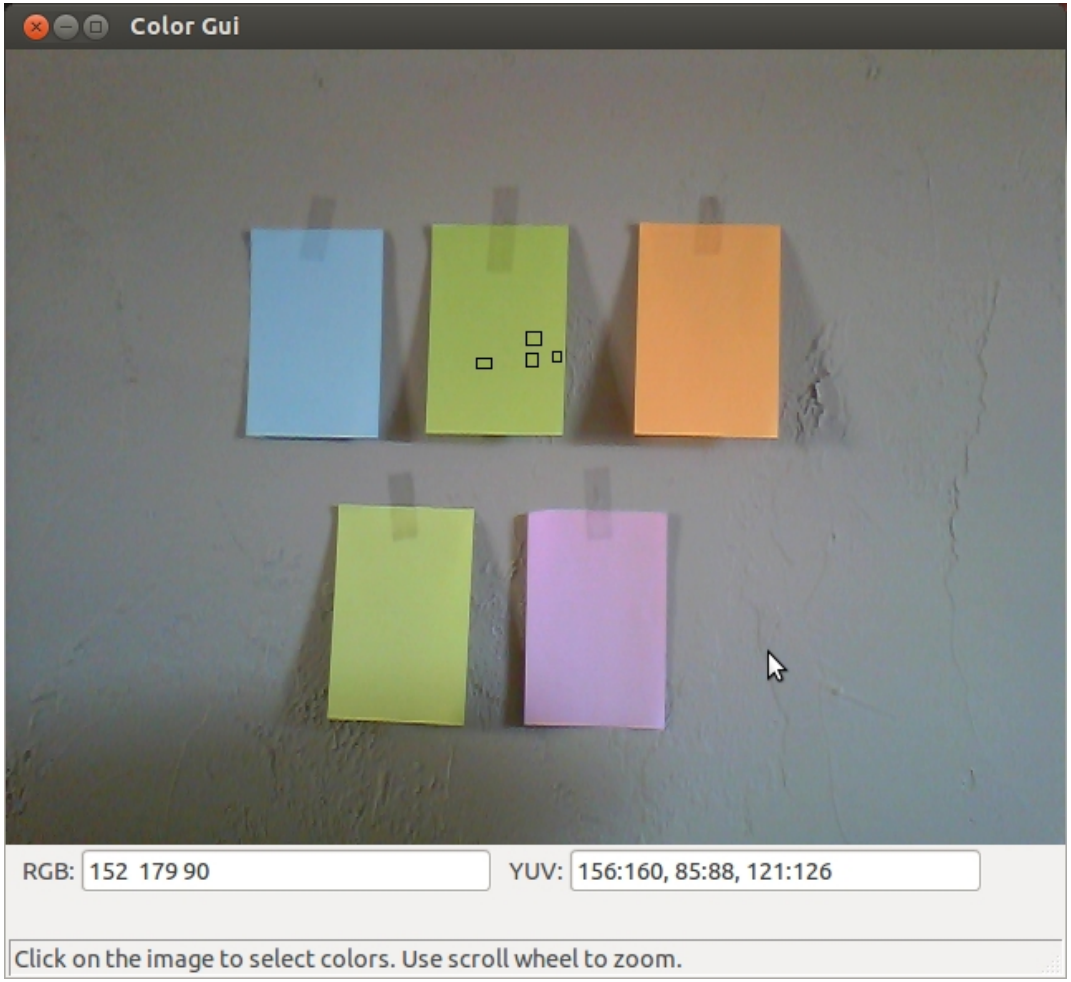
```
rosrun cmvision colorgui image:=/v4l/camera/image_raw
```

- Note that the data in topic `/v4l/camera/image_raw` is published by `gscam` node
- A windows will pop-up and shows the image from your camera

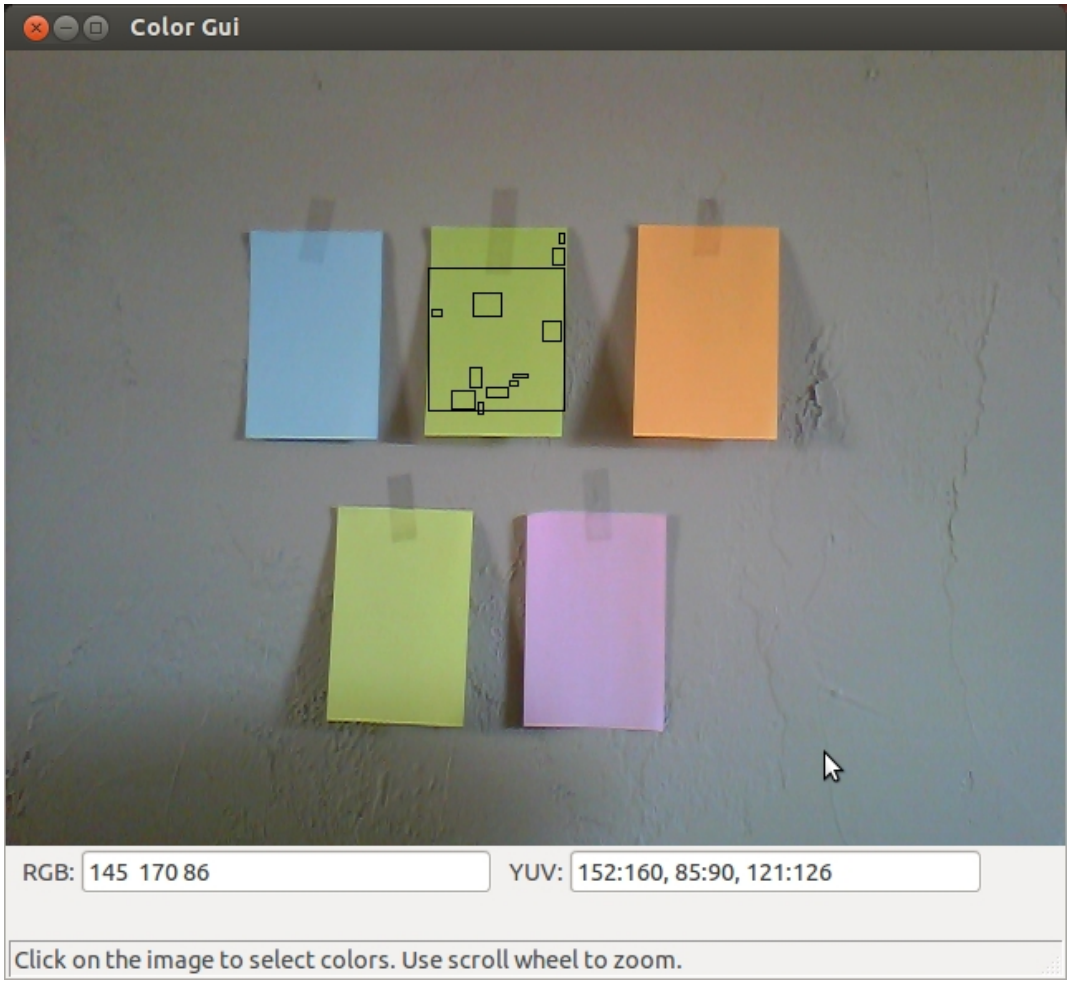
- After clicking on the object in the image couple times, you may see a set of rectangles
- Each rectangle represents a detected blob
- Move object around to see whether it can detect the object
- If it cannot detect, click on the object couple more times to increase the range of detecting color
- Information at the bottom (YUV) will be use to set the color to be detected

colorgui Example

- First couple clicks

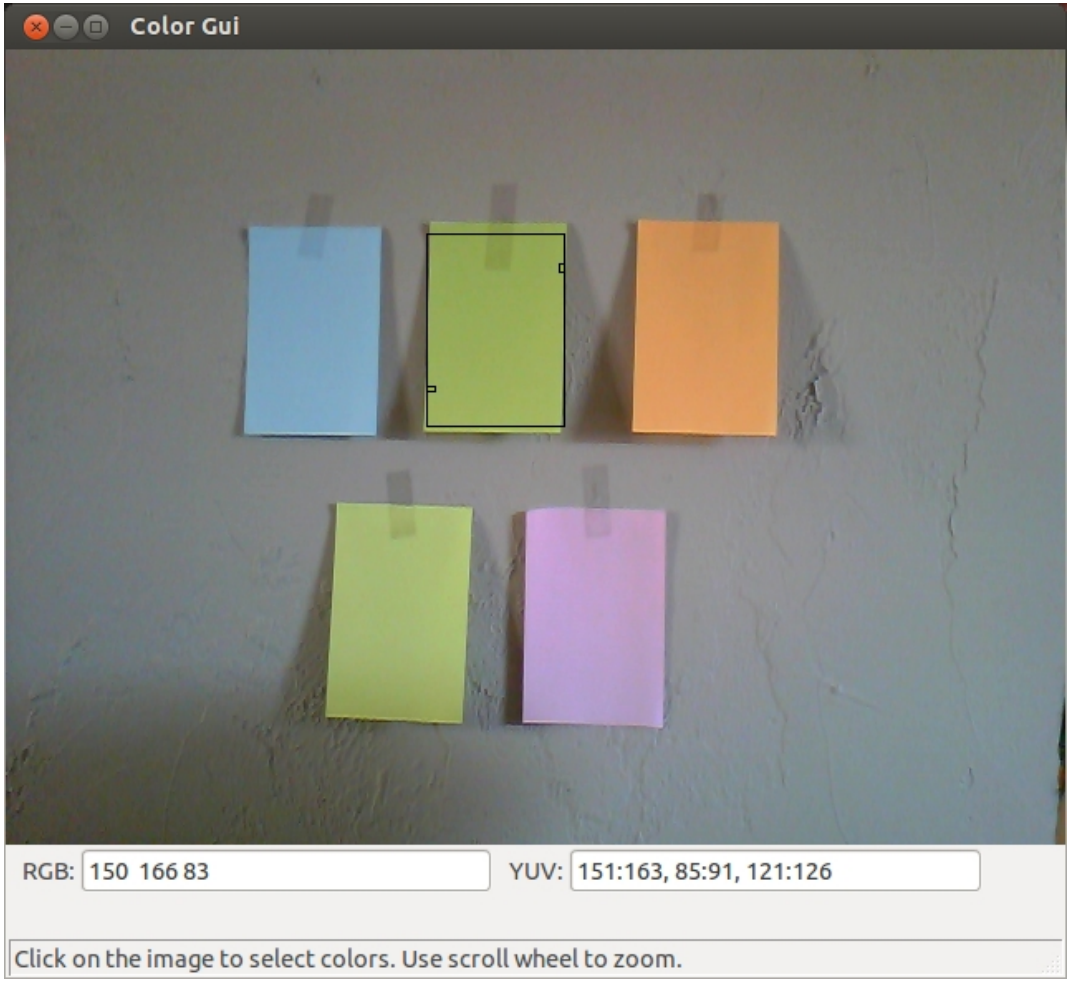


- Couple more clicks around the desired object



`colorgui` Example

- Couple more and take note about RGB and YUV



colors.txt

- For `cmvision` node, color information should be stored in a text file
- By default, the name of the text file is `colors.txt`
- A color file consists of two parts `[colors]` and `[thresholds]` as shown below:

```
[colors]
(150, 166, 83) 0.000000 1 BrightGreen

[thresholds]
(151:163, 85:91, 121:126)
```

- `cmvision` node can be used to detect more than one color by adding more lines for `[colors]` and `[thresholds]`

```
[colors]
(150, 166, 83) 0.000000 1 BrightGreen
(0, 255, 0) 0.000000 2 Green

[thresholds]
(151:163, 85:91, 121:126)
(47:87, 148:162, 93:113)
```

colors.txt

- Consider the following color file:

```
[colors]
(150, 166, 83) 0.000000 1 BrightGreen
(0, 255, 0) 0.000000 2 Green

[thresholds]
(151:163, 85:91, 121:126)
(47:87, 148:162, 93:113)
```

- Each line under `[colors]` consists of the following:
  - `(R, G, B)`: The red, green, and blue values of rectangles that are used to blob region
  - `0.000000`: Not being used any more (I guess)
  - `1`: Not being used any more (I guess)
  - `BrightGreen`: The name of the blob which can be used to distinguish among multiple detected blobs

colors.txt

- Consider the following color file:

```
[colors]
(150, 166, 83) 0.000000 1 BrightGreen
(0, 255, 0) 0.000000 2 Green

[thresholds]
(151:163, 85:91, 121:126)
(47:87, 148:162, 93:113)
```

- Each line under `[thresholds]` consists of the following:
  - `(minY:maxY, minU:maxU, minV:maxV)`: The YUV threshold from `colorgui`
- Note that the first line of `colors` is associated with the first line of `thresholds`, and so on

cmvision.launch

- Parameters of `cmvision` are needed to be set
- First, copy the default launch file:

```
/home/student/cs1567/src/cmvision/cmvision.launch
```

  to your `../mypackage` directory
- In the copied file, change the followings:
  - Under `param name="cmvision/color_file"`, change its `value` to

```
value="/home/student/cs1567/src/mypackage/colors.txt"
```

  - If you put `colors.txt` in different directory, adjust it accordingly
  - Under `param name="cmvision/debug_on"`, change the `value` to
    - `value="true"` if you want to see image and detect blobs while `cmvision` is running
    - `value="false"` otherwise
  - Under `node name="cmvision"` change the `args` to

```
args="image:=/v4l/camera/image_raw"
```

Your `cmvision.launch` should look like the following:

```
<launch>

  <!-- Location of the cmvision color file -->
  <param name="cmvision/color_file" type="string"
         value="/home/student/cs1567/src/mypackage/colors.txt" />

  <!-- Turn debug output on or off -->
  <param name="cmvision/debug_on" type="bool" value="true"/>

  <!-- Turn color calibration on or off -->
  <param name="cmvision/color_cal_on" type="bool" value="false"/>

  <!-- Enable Mean shift filtering -->
  <param name="cmvision/mean_shift_on" type="bool" value="false"/>

  <!-- Spatial bandwidth: Bigger = smoother image -->
  <param name="cmvision/spatial_radius_pix" type="double" value="2.0"/>

  <!-- Color bandwidth: Bigger = smoother image-->
  <param name="cmvision/color_radius_pix" type="double" value="40.0"/>

  <node name="cmvision" pkg="cmvision" type="cmvision" args="image:=/v4l/camera/image_raw"
        output="screen" />
</launch>
```

Blobs.msgs

- `cmvision` node publishes a message of type `Blobs`
- Again, import it first:

```
from cmvision.msg import Blobs
```

- A message of type `Blobs` consists of various components:
  - `header`: Message header
  - `image_width`: The width of the image in pixels
  - `image_height`: The height of the image in pixels
  - `blob_count`: The number of detected blobs
  - `blobs`: An array of data of type `Blob`
- **Note**: `Blob` (with out `s`) is another type of message

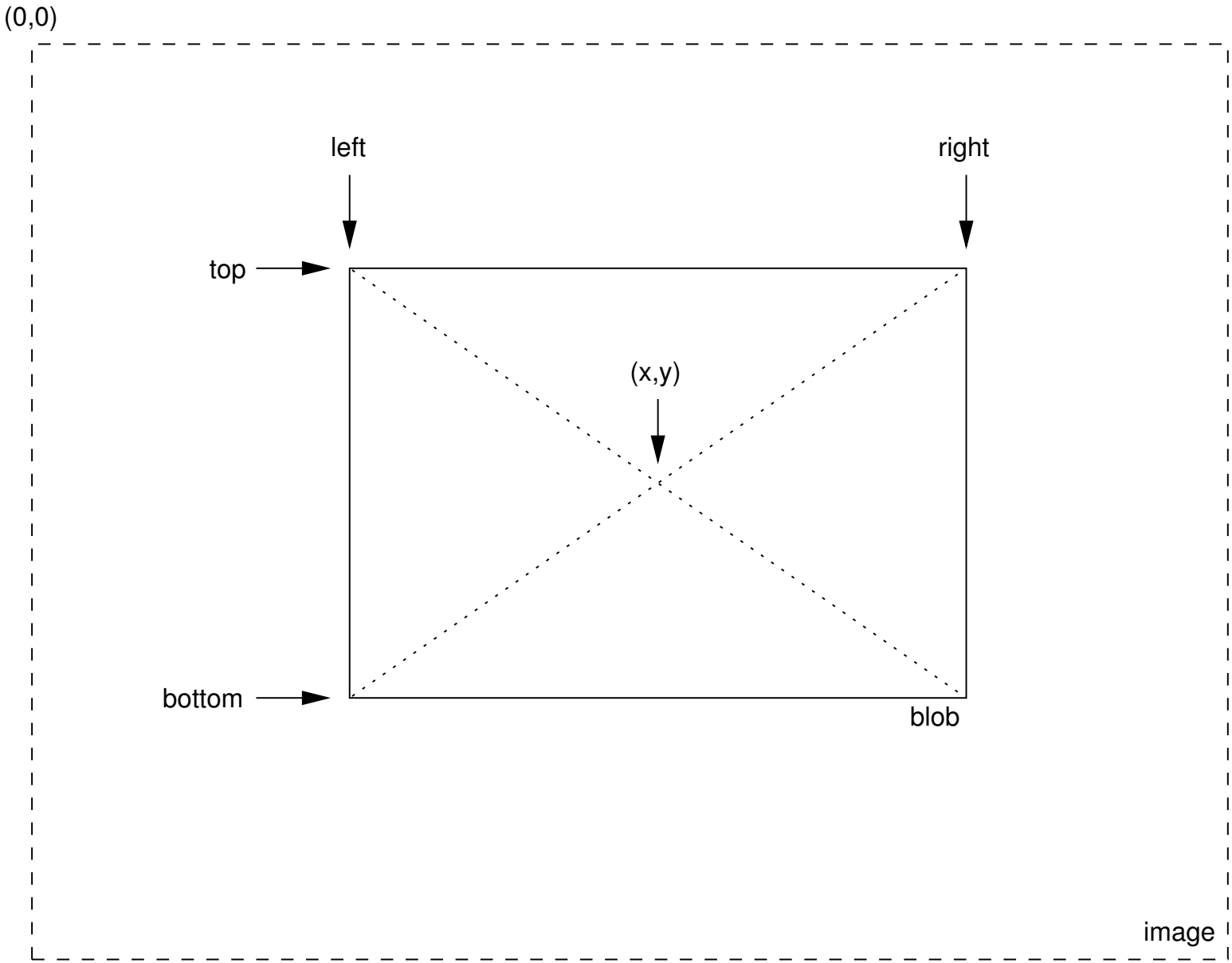
Blob.msgs

- `Blob.msgs` stores information about each blob detected by `cmvision` node
- Must be explicitly imported:

```
from cmvision.msg import Blobs, Blob
```

- A message of type `Blob` consists of various components:
  - `name`: The name of color/object specified in `colors.txt` (`String`)
  - `red`: The value of red
  - `green`: The value of green
  - `blue`: The value of blue
  - `area`: The area of the blob (number of pixels)
  - `x`: The x coordinate of the center of the blob
  - `y`: The y coordinate of the center of the blob
  - `left`: The x coordinate of the left side of the region
  - `right`: The x coordinate of the right side of the region
  - `top`: The y coordinate of the top side of the region
  - `bottom`: The y coordinate of the bottom side of the region

Blob.msgs



Problem Solving Ideas

- How to filter out environment?
- How do we know that two or more blobs are on the same object?
  - Fixed size objects
  - Variable size objects
- If we know that these 10 blobs are on the same object, how to find the center of the object?
- How do we know that there are two or more objects of the same color?
- How to filter out noise from camera?

Visualize Blobs using OpenCV

- Sometimes we need to process blobs:
  - Filter out blobs that are too smalls
  - Filter out blobs that are outside of region of interest
  - Merge multiple blobs in to one
- We may need to see a result in an image
- OpenCV provides libraries for Computer Vision:
  - Show image/video on screen
  - Mouse and keyboard events
  - Draw lines, rectangles, string on image
  - Foreground/background filtering

Visualize Blobs using OpenCV

```
#!/usr/bin/env python
import roslib
import rospy
import cv2
import copy
from sensor_msgs.msg import Image
from cv_bridge import CvBridge, CvBridgeError
from cmvision.msg import Blobs, Blob

colorImage = Image()
isColorImageReady = False
blobsInfo = Blobs()
isBlobsInfoReady = False

def updateColorImage(data):
    global colorImage, isColorImageReady
    colorImage = data
    isColorImageReady = True

def updateBlobsInfo(data):
    global blobsInfo, isBlobsInfoReady
    blobsInfo = data
    isBlobsInfoReady = True
```

Visualize Blobs using OpenCV

```
def main():
    global colorImage, isColorImageReady, blobsInfo, isBlobsInfoReady
    rospy.init_node('showBlobs', anonymous=True)
    rospy.Subscriber("/blobs", Blobs, updateBlobsInfo)
    rospy.Subscriber("/v4l/camera/image_raw", Image, updateColorImage)
    bridge = CvBridge()
    cv2.namedWindow("Blob Location")

    while not rospy.is_shutdown() and
          (not isBlobsInfoReady or not isColorImageReady):
        pass

    while not rospy.is_shutdown():
        :
```

Visualize Blobs using OpenCV

```
def main():
        :
    while not rospy.is_shutdown():
        try:
            color_image = bridge.imgmsg_to_cv2(colorImage, "bgr8")
        except CvBridgeError, e:
            print e
            print "colorImage"

        blobsCopy = copy.deepcopy(blobsInfo)

        for b in blobsCopy.blobs:
            cv2.rectangle(color_image, (b.left, b.top),
                          (b.right, b.bottom), (0,255,0), 2)
        cv2.imshow("Color Image", color_image)
        key = cv2.waitKey(1)

        if key == ord('q'):
            break

    cv2.destroyAllWindows()

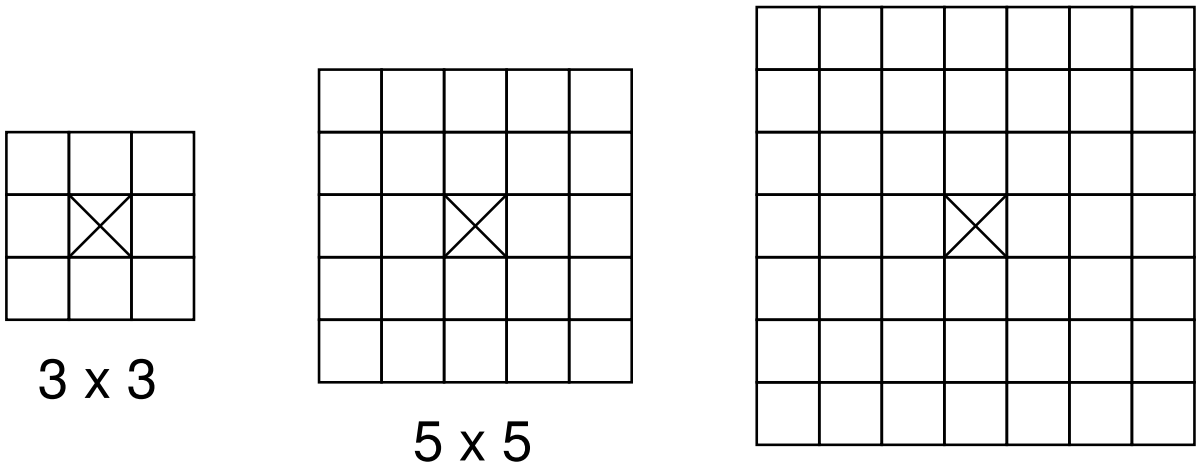
if __name__ == '__main__':
    main()
```

Blob Detection Algorithm

- Given an image, the following steps are generally used:
  1. Blur the image to reduce noise
  2. Filter out all pixels that are out-of-range
  3. Find groups of pixels that are connected to each other

Gaussian Blur

- One of the most popular blur algorithm is called Gaussian blur



- The new RGB values of the pixel at center is an **weight** average of RGB of pixels around it
- The weight of each pixel is based on the Gaussian normal distribution

Filter

- Filter can be done based on RGB or YUV
  - Recall that `cmvision` uses YUV
- Once a filter is applied, we should get a one bit image (black and white)
  - We do not need information about color of each pixel anymore
  - Only need to know which pixels are in a given range
- Example:

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	1	1	0	0	0	0	0	0
0	0	0	0	0	1	1	1	0	0	0	0	0	0
0	0	0	0	0	1	1	1	0	0	1	1	0	0
0	0	0	0	0	0	1	0	0	0	0	1	1	0
0	1	1	1	0	0	1	0	0	0	0	0	0	0
0	0	1	1	0	0	1	0	0	0	0	0	0	0
0	0	0	0	0	1	1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

- For simplicity, assume that a blob is a 4-tuple

```
(left, right, top, bottom)
```

A Blob Detection Algorithm

- First, we need to search for the first pixel that are non-zero
  - Scan left to right and top to bottom

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	1	1	0	0	0	0	0	0
0	0	0	0	0	1	1	1	0	0	0	0	0	0
0	0	0	0	0	1	1	1	0	0	1	1	0	0
0	0	0	0	0	0	1	0	0	0	0	1	1	0
0	1	1	1	0	0	1	0	0	0	0	0	0	0
0	0	1	1	0	0	1	0	0	0	0	0	0	0
0	0	0	0	0	1	1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

- Once the first pixel is found:
  1. Record this coordinate so that we can restart searching for the next blob
  2. Construct a new blob (data)
  3. Left and right are 5
  4. Top and bottom are 1
  5. Center, width, and height of the blob will be calculated later
- Now, our first blob is `(5, 5, 1, 1)`

A Blob Detection Algorithm

- Next, set the pixel to zero and scanning for non-zero neighbor
- If a non-zero neighbor is found:
  - Adjust the blob accordingly
  - Add coordinate `(x,y)` of the non-zero neighbor into a queue
  - Change non-zero neighbors to 0

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	1	1	1	0	0	1	1	0	0
0	0	0	0	0	0	1	0	0	0	0	1	1	0
0	1	1	1	0	0	1	0	0	0	0	0	0	0
0	0	1	1	0	0	1	0	0	0	0	0	0	0
0	0	0	0	0	1	1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

- Currently
  - Blob is `(5, 6, 1, 2)`
  - Queue is `[(6,1), (5,2), (6,2)]`

A Blob Detection Algorithm

- Repeat until the queue is empty:
  1. Dequeue the first coordinate
  2. Scan for its non-zero neighbor
  3. Add all coordinates `(x,y)` of the non-zero neighbor into a queue
  4. Change all non-zero neighbor to 0
  5. Go back to step 1
- Once the queue is empty, we get the first blob `(5, 7, 1, 7)`

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	1	0	0
0	0	0	0	0	0	0	0	0	0	0	1	1	0
0	1	1	1	0	0	0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

Blob Detection using OpenCV

- First, we need to blur an image using the `GaussianBlur()` function
- The following code blurs an image and display:

```
import cv2

def main():
    color_image = cv2.imread('start_img.jpg', cv2.IMREAD_COLOR)
    blur_image = cv2.GaussianBlur(color_image, (5, 5), 0)
    cv2.imshow('Image', color_image)
    cv2.imshow('Blur Image', blur_image)
    while True:
        key = cv2.waitKey(0) & 0xFF
        if key == ord('q'):
            break;
    cv2.destroyAllWindows()

if __name__ == '__main__':
    main()
```

- The parameter `(5, 5)` specifies how many pixels to be used (must be odd numbers)

Convert RGB to YUV

- `OpenCV` supplies the `cvtColor()` function to convert one color spectrum to another

```
import cv2

def main():
    color_image = cv2.imread('../start_img.jpg', cv2.IMREAD_COLOR)
    blur_image = cv2.GaussianBlur(color_image, (5, 5), 0)
    yuv_image = cv2.cvtColor(blur_image, cv2.COLOR_BGR2YUV)
    y, u, v = cv2.split(yuv_image)
    cv2.imshow('Image', blur_image)
    cv2.imshow('Y', y)
    cv2.imshow('U', u)
    cv2.imshow('V', v)
    while True:
        key = cv2.waitKey(0) & 0xFF
        if key == ord('q'):
            break;
    cv2.destroyAllWindows()

if __name__ == '__main__':
    main()
```

Filter Unwanted Pixels

- Unwanted pixels can be filtered out using the `inRange()` function:

```
def main():
    color_image = cv2.imread('../start_img.jpg', cv2.IMREAD_COLOR)
    blur_image = cv2.GaussianBlur(color_image, (5, 5), 0)

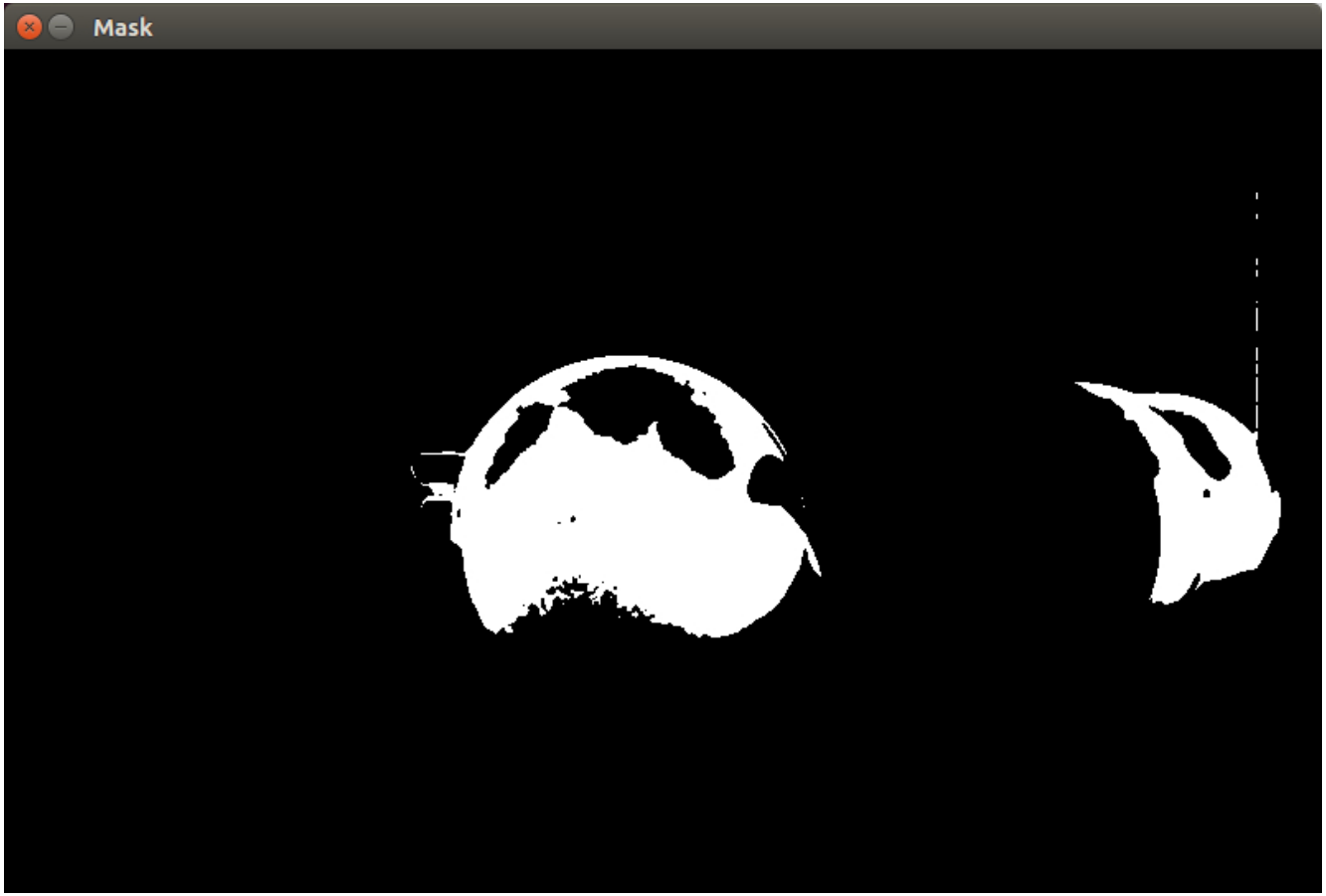
    cv2.namedWindow('Mask')
    mask_image = cv2.inRange(blur_image, lower_bound, upper_bound);
    cv2.imshow('Mask', mask_image)
    cv2.waitkey(0);
    cv2.destroyAllWindows()

if __name__ == '__main__':
    main()
```

- The `lower_bound` and `upper_bound` are `(B, G, R)` in this case

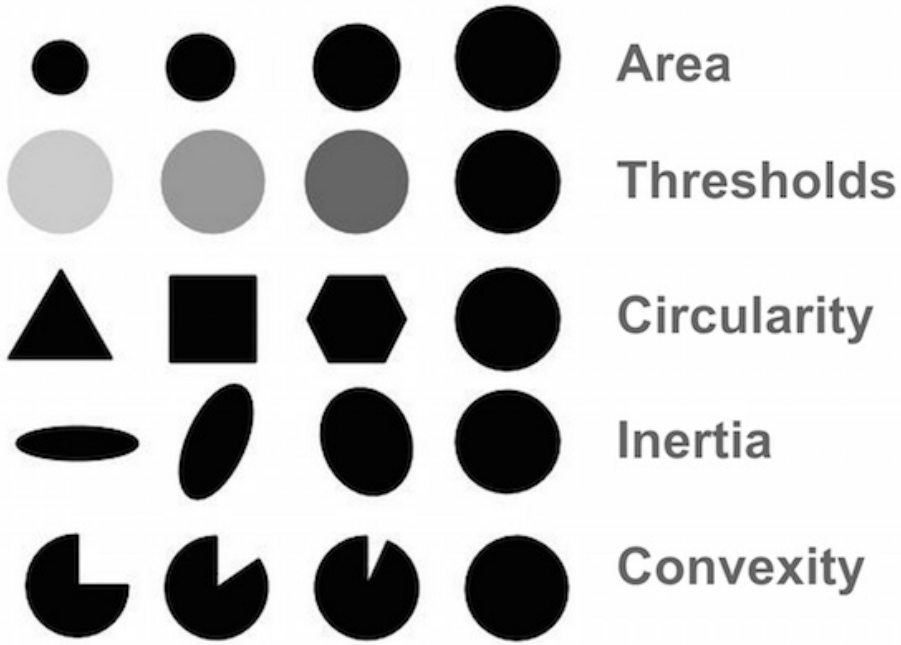
Filter Unwanted Pixels

- Masked Image:



SimpleBlobDetector()

- `OpenCV` supplies the `SimpleBlobDetector()` function which detects blobs based on a given parameter:



SimpleBlobDetector() Parameters

- First we need to set the parameter:

```
params = cv2.SimpleBlobDetector_Params()
params.minThreshold = 0
params.maxThreshold = 255
params.filterByArea = True
params.minArea = 50
params.maxArea = 256 * 256
params.filterByCircularity = False
params.minCircularity = 0.1
params.filterByConvexity = False
params.minConvexity = 0.9
params.filterByInertia = False
params.minInertiaRatio = 0.5
ver = (cv2.__version__).split('.')
if int(ver[0]) < 3:
    detector = cv2.SimpleBlobDetector(params)
else:
    detector = cv2.SimpleBlobDetector_create(params)
```

SimpleBlobDetector()

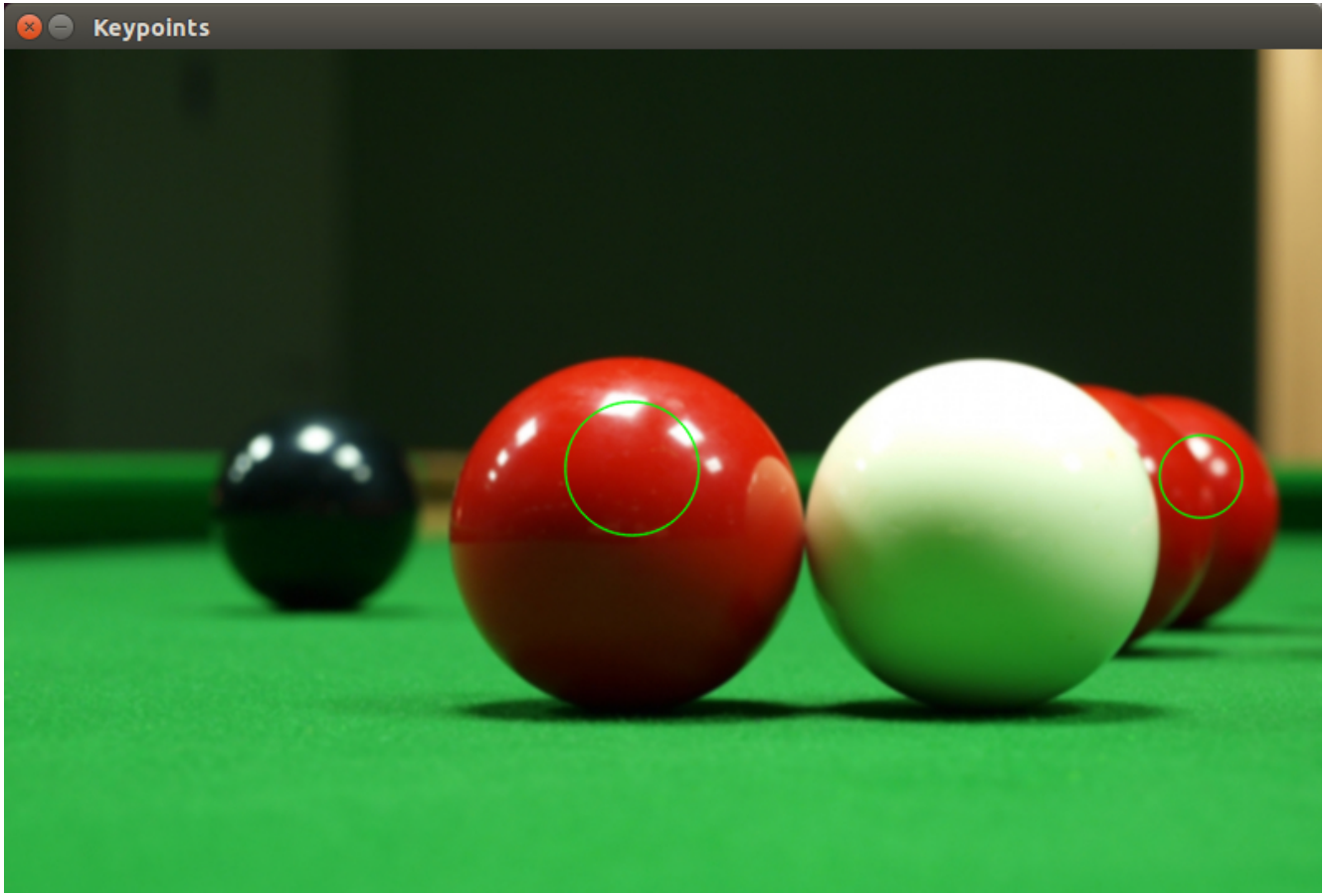
- Then we can use the `detector` object to detect blobs from masked image

```
keypoints = detector.detect(255 - mask_image)
kp_image =
    cv2.drawKeypoints(blur_image,
                      keypoints,
                      np.array([]),
                      (0, 255, 0),
                      cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv2.imshow('Keypoints', kp_image)
```

- To extract blobs information, simply look at the `keypoints` variable

SimpleBlobDetector()

- Result from the `drawKeypoints()` function to the original image:



SimpleBlobDetector()

- The `SimpleBlobDetector()` function is good for round object
- It may not perform well for our application (high CPU utilization)
- With a large blob, it only show a small keypoint
- Parameters are hard to understand

Blob Detection using Contour

- Another method is to detect contours of masked image
- A contour is a curve joining all the continuous points (along the boundary), having same color or intensity
- The contours are a useful tool for shape analysis and object detection and recognition
- In `OpenCV`, the function `findContours()` is given

findContours()

- `findContours()` example:

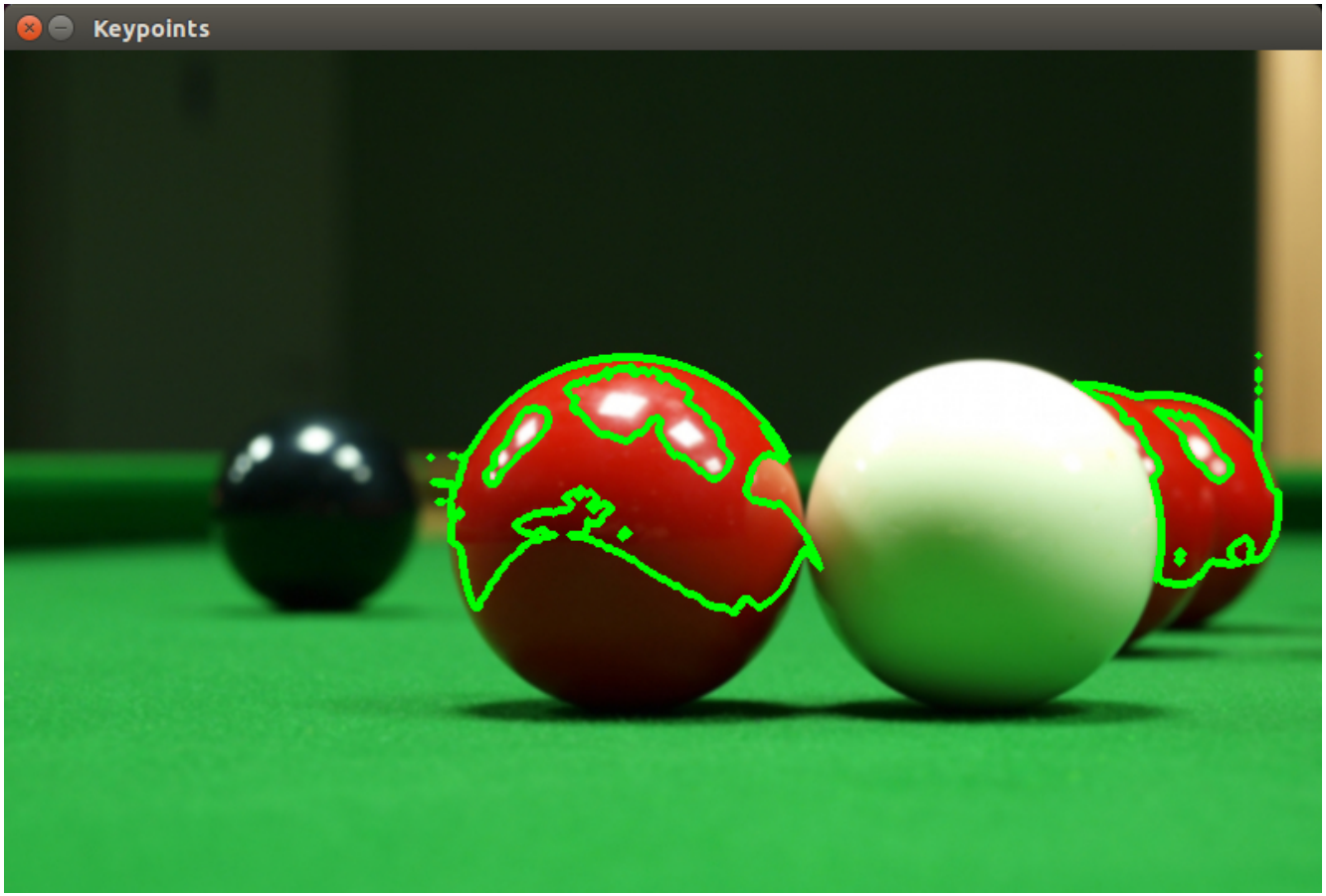
```
def main():
    color_image = cv2.imread('../start_img.jpg', cv2.IMREAD_COLOR)
    blur_image = cv2.GaussianBlur(color_image, (5, 5), 0)

    cv2.namedWindow('Contour Image')
    (cnts, _) = cv2.findContours(mask_image.copy(), cv2.RETR_TREE,
                                 cv2.CHAIN_APPROX_SIMPLE)
    cv2.drawContours(blur_image, cnts, -1, (0,255,0), 3)
    cv2.imshow('Contour Image', blur_image)
    cv2.waitKey(0);
    cv2.destroyAllWindows()

if __name__ == '__main__':
    main()
```

drawContours()

- Use the `drawContours()` to draw all contours into the original image



- Note that there are contours inside other contours
- We need bounding box for each contour and eliminate contours that are inside another

findContours() with hierarchy

- Filter out some contours

```
def main():
    color_image = cv2.imread('../start_img.jpg', cv2.IMREAD_COLOR)
    blur_image = cv2.GaussianBlur(color_image, (5, 5), 0)

    cv2.namedWindow('Bounding')
    (cnts, hierarchy) = cv2.findContours(mask_image.copy(),
                                         cv2.RETR_TREE,
                                         cv2.CHAIN_APPROX_SIMPLE)
    for i in range(0, len(cnts)):
        c = cnts[i]
        if cv2.contourArea(c) < 100:
            continue
        if not hierarchy[0][i][3] == -1:
            continue
        x,y,w,h = cv2.boundingRect(c)
        cv2.rectangle(blur_image, (x,y), (x + w, y + h),
                      (0, 255, 0), 3);
    cv2.imshow('Bounding', blur_image)
    cv2.waitKey(0);
    cv2.destroyAllWindows()

if __name__ == '__main__':
    main()
```

drawContours()

- After filtering out small contours and those that are inside another

